RActividad 1: Módulo 1

1) Importamos la base de datos correspondiente.

```
# Preprocesado y modelado
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from numpy import linalg as LA
```

```
[6]: # Cargar la base de datos Iris
df_original = pd.read_csv("C:/Users/gus6e/OneDrive/Downloads/iris.csv")

df_original
```

[6]:

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Virginica
146	6.3	2.5	5.0	1.9	Virginica
147	6.5	3.0	5.2	2.0	Virginica
148	6.2	3.4	5.4	2.3	Virginica
149	5.9	3.0	5.1	1.8	Virginica

150 rows × 5 columns

Después se realiza la estandarización de las variables obteniendo valores transformados para cada observación, lo que permite que las variables queden en una escala comparable antes de realizar el PCA.

```
[8]: # Determinar el número de renglones de la base de datos
index = df.index
renglones = len(index)

renglones
```

```
[8]: 150
```

```
[9]: # Estandarización de las variables
df2 = StandardScaler().fit_transform(df)

df2
```

```
[9]: array([[-9.00681170e-01,  1.01900435e+00, -1.34022653e+00,
        -1.31544430e+00],
       [-1.14301691e+00, -1.31979479e-01, -1.34022653e+00,
        -1.31544430e+00],
       [-1.38535265e+00,  3.28414053e-01, -1.39706395e+00,
        -1.31544430e+00],
       [-1.50652052e+00,  9.82172869e-02, -1.28338910e+00,
        -1.31544430e+00],
       [-1.02184904e+00,  1.24920112e+00, -1.34022653e+00,
        -1.31544430e+00],
       [-5.37177559e-01,  1.93979142e+00, -1.16971425e+00,
        -1.05217993e+00],
       [-1.50652052e+00,  7.88807586e-01, -1.34022653e+00,
        -1.18381211e+00],
       [-1.02184904e+00,  7.88807586e-01, -1.28338910e+00,
        -1.31544430e+00],
       [-1.74885626e+00, -3.62176246e-01, -1.34022653e+00,
        -1.31544430e+00],
       [-1.14301691e+00,  9.82172869e-02, -1.28338910e+00,
```

Se calcula la matriz de correlaciones de las variables estandarizadas, después se entrena el modelo de análisis de componentes principales utilizando los datos escalados. Se calculan los eigenvalores.

```
[10]: # Calcular la matriz de correlaciones para la matriz transformada
A = (1/renglones) * np.dot(df2.T, df2)

A
```

```
[10]: array([[ 1.        , -0.11756978,  0.87175378,  0.81794113],
       [-0.11756978,  1.        , -0.4284401 , -0.36612593],
       [ 0.87175378, -0.4284401 ,  1.        ,  0.96286543],
       [ 0.81794113, -0.36612593,  0.96286543,  1.        ]])
```

```
[11]: # Entrenamiento del modelo de análisis de componentes principales
# con escalado de los datos

pca_pipe = make_pipeline(
    StandardScaler(),
    PCA()
)

pca_pipe.fit(df)

# Se extrae el modelo entrenado del pipeline
modelo_pca = pca_pipe.named_steps["pca"]
```

```
[12]: # Eigenvalores

print("Eigenvalores")

results = LA.eigvals(A)

print(results)
```

```
Eigenvalores
[2.91849782 0.91403047 0.14675688 0.02071484]
```

2) Los dos primeros componentes explican en conjunto aproximadamente el 95.81 % de la variabilidad total, por lo que son suficientes para representar la mayor parte de la información de la base de datos.

[13]:
```
# Porcentaje de la varianza explicada por cada componente

print("Porcentaje de varianza explicada por componente")

print(modelo_pca.explained_variance_ratio_)
```

```
Porcentaje de varianza explicada por componente
[0.72962445 0.22850762 0.03668922 0.00517871]
```

[14]:
```
# Porcentaje de varianza explicada por los dos primeros componentes

varianza_dos_componentes = (
    modelo_pca.explained_variance_ratio_[0] +
    modelo_pca.explained_variance_ratio_[1]
) * 100

print("Varianza explicada por los dos primeros componentes:")
print(varianza_dos_componentes, "%")
```

```
Varianza explicada por los dos primeros componentes:
95.81320720000166 %
```

Cálculo de los eigenvectores.

```
# Eigenvectores

print("Eigenvectores por renglón:")

componentes = pd.DataFrame(
    data=modelo_pca.components_,
    columns=df.columns,
    index=["PC1", "PC2", "PC3", "PC4"]
)

componentes
```

Eigenvectores por renglón:

	sepal.length	sepal.width	petal.length	petal.width
PC1	0.521066	-0.269347	0.580413	0.564857
PC2	0.377418	0.923296	0.024492	0.066942
PC3	0.719566	-0.244382	-0.142126	-0.634273
PC4	-0.261286	0.123510	0.801449	-0.523597

Se calculan las proyecciones de cada observación sobre los cuatro componentes principales mediante los eigenvectores obtenidos del modelo PCA. Después, los resultados se organizan en un dataframe con las columnas PC1, PC2, PC3 y PC4. Posteriormente, se agrega la variable variety para identificar a qué especie pertenece cada observación.

[16]:
```
# Proyecciones de las observaciones en los componentes principales

proyecciones = np.dot(
    modelo_pca.components_,
    df2.T
)

proyecciones = pd.DataFrame(
    proyecciones,
    index=["PC1", "PC2", "PC3", "PC4"]
)

proyecciones = proyecciones.transpose()

proyecciones
```

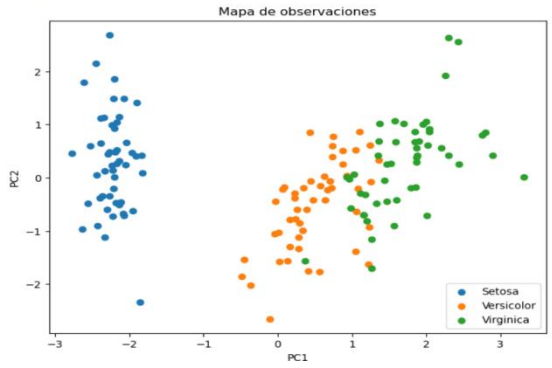
[16]:

	PC1	PC2	PC3	PC4
0	-2.264703	0.480027	0.127706	-0.024168
1	-2.080961	-0.674134	0.234609	-0.103007
2	-2.364229	-0.341908	-0.044201	-0.028377
3	-2.299384	-0.597395	-0.091290	0.065956

Se genera el mapa de observaciones utilizando los dos primeros componentes principales, PC1 y PC2, diferenciando las especies por color. Se observa que Setosa queda claramente separada del resto, mientras que Versicolor y Virginica presentan mayor cercanía y cierta superposición. Esto indica que los dos primeros componentes permiten distinguir adecuadamente la estructura de agrupación de las especies.

```
ax.set_ylabel('PC2')
ax.legend()

plt.show()
```



Se extraen únicamente los dos primeros componentes principales, PC1 y PC2, para construir el mapa de factores.

[19]:
```
# Obtener únicamente los dos primeros componentes
# para elaborar el mapa de factores

componentes_2 = pd.DataFrame(
    data=modelo_pca.components_,
    columns=df.columns,
    index=["PC1", "PC2", "PC3", "PC4"]
)

componentes_2 = componentes_2.iloc[0:2, :]

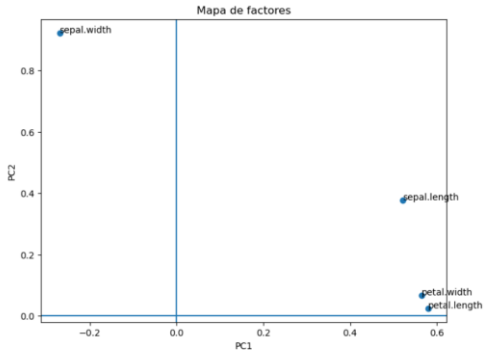
componentes_2 = componentes_2.T

componentes_2
```

[19]:

	PC1	PC2
sepal.length	0.521066	0.377418
sepal.width	-0.269347	0.923296
petal.length	0.580413	0.024492
petal.width	0.564857	0.066942

Se construye el mapa de factores utilizando PC1 y PC2. Se observa que petal.length y petal.width están muy próximas entre sí y se relacionan principalmente con PC1. sepal.length también tiene una relación positiva con PC1, mientras que sepal.width se asocia principalmente con PC2.



Se observa que Setosa presenta los valores más bajos en petal.length y petal.width, Versicolor valores intermedios y Virginica los valores más altos. Esto respalda la separación observada en el mapa de observaciones, especialmente a lo largo de PC1.

```
df_original.groupby("variety").head(3)
```

21]:

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
50	7.0	3.2	4.7	1.4	Versicolor
51	6.4	3.2	4.5	1.5	Versicolor
52	6.9	3.1	4.9	1.5	Versicolor
100	6.3	3.3	6.0	2.5	Virginica
101	5.8	2.7	5.1	1.9	Virginica
102	7.1	3.0	5.9	2.1	Virginica

22]:

```
# Promedio de las variables para cada especie
df_original.groupby("variety").mean()
```

22]:

variety	sepal.length	sepal.width	petal.length	petal.width
Setosa	5.006	3.428	1.462	0.246
Versicolor	5.936	2.770	4.260	1.326
Virginica	6.588	2.974	5.552	2.026

3) Al comparar el mapa de observaciones con el mapa de factores, se observa que la separación de las especies ocurre principalmente sobre PC1. En el mapa de factores, petal.length y petal.width presentan una fuerte asociación con este componente. Esto indica que las diferencias en el tamaño de los pétalos contribuyen de manera importante a la separación observada entre las especies, especialmente entre Setosa y las demás.